Algoritmos y Estructuras de Datos

Guía Práctica 7
Diseño y Elección de Estructuras
Segundo Cuatrimestre 2026



Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Diseño sobre listas

Ejercicio 1. Implemente el TAD ConjuntoAcotado⟨T⟩ usando la siguiente estructura.

```
TAD ConjAcotado⟨T⟩ {
    obs elems : conj⟨T⟩
    obs capacidad : ℤ
    proc conjVacío(in capacidad : ℤ) : ConjuntoAcotado⟨T⟩ {
        requiere { capacidad > 0 }
        asegura { res.elems = ⟨⟩ ∧ res.capacidad = capacidad }
    }
    proc igualdad(in C1 : ConjuntoAcotado⟨T⟩, in C2 : ConjuntoAcotado⟨T⟩) : bool {
        asegura {
            res = true ↔ |C1.elems| = |C2.elems| ∧L
                 (∀e : T) (e ∈ C1.elems ↔ e ∈ C2.elems[i]) ∧
            C1.capacidad = C2.capacidad
        }
    }
    proc pertenece(in c : ConjAcotado⟨T⟩, in e : T) : bool {
        asegura { res = true ↔ e ∈ c.elems }
    }
    proc agregar(inout c : ConjAcotado⟨T⟩, in e : T) {
        requiere { c = C0 ∧ |c.elems| < c.capacidad }
        asegura { c.elems = C0.elems ∪ ⟨e⟩ ∧ c.capacidad = C0.capacidad }
    }
    proc sacar(inout c : Conjunto⟨T⟩, in e : T) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems − ⟨e⟩ ∧ c.capacidad = C0.capacidad }
    }
    proc unir(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 ∧ |c.elems ∪ c'.elems| < c.capacidad }
        asegura { c.elems = C0.elems ∪ c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc restar(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems − c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc intersecar(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems ∩ c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc tamaño(in c : Conjunto⟨T⟩) : ℤ {
        asegura { res = |c.elems| }
    }
    proc capacidad(in c : Conjunto⟨T⟩) : ℤ {
        asegura { res = c.capacidad }
    }
}
```

Módulo ConjuntoArr<T> **implementa** ConjuntoAcotado$\langle T \rangle$ <

```
    var datos: Vector<T>
    var tamaño: int
```

>

a) Escriba el invariante de representación y la función de abstracción

b) Escriba los algoritmos para las operaciones

i) **conjVacío**
ii) **pertence**
iii) **agregar**
iv) **sacar**
v) **unir**
vi) **intersecar**

c) Calcule la complejidad de cada una de esas operaciones

d) ¿Qué cambios haría en su implementación si se quiere que la operación `agregar` sea lo más rápida posible? ¿Y si se quiere acelerar la operación `pertenece`? Indique los cambios en la estructura, el invariante de representación, la función de abstracción y los algoritmos.

Solución

a) `pred InvRep`(ci : ConjuntoArr<T>) {

$$ci.tamaño \leq |ci.datos| \wedge_L$$
$$(\forall i, j : \mathbb{Z})\ (0 \leq i, j < |ci.tamaño| \rightarrow_L (ci.datos[i] = ci.datos[j] \leftrightarrow i = j))$$

}

`func Abs`(ci : ConjuntoArr<T>) : $ConjuntoAcotado$ {

$$ct : ConjuntoAcotado \mid$$
$$ct.capacidad = |ci.datos.elems| \wedge$$
$$|ct.elems| = ci.tamaño \wedge_L$$
$$(\forall e : T)\ (e \in ct.elems \leftrightarrow e \in ci.datos.elems)$$

}

b)
```
proc conjVacío<T>(int capacidad): ConjuntoArr<T>
    res := new ConjuntoArr<T>
    res.datos := new Vector<T>[capacidad]
    res.tamaño := 0

    return res
endproc

proc pertence(e: T, c: ConjuntoArr<T>): bool
    res := false
    i := 0

    while (i < c.tamaño)
        res := c.datos[i] = e
        i := i + 1
    endwhile

    return res
endproc

proc agregar(e: T, c: ConjuntoArr<T>)
    c.datos[c.tamaño] := e
```

```
    c.tamaño := c.tamaño + 1
endproc

proc sacar(e: T, c: ConjuntoArr<T>)
    i := 0

    while (c.datos[i] != e)
        i := i + 1
    endwhile

    j := i

    while (j < c.tamaño - 1)
        c.datos[j] := c.datos[j - 1]
        j := j + 1
    endwhile

    c.tamaño := c.tamaño - 1
endproc

proc unir(c: ConjuntoArr<T>, otro: ConjuntoArr<T>)
    i := 0
    while (i < otro.tamaño)
        if (! pertence(otro.datos[i], c))
            agregar(otro.datos[i], c)
        endif
        i := i + 1
    endwhile
endproc

proc intersecar(c: ConjuntoArr<T>, otro: ConjuntoArr<T>)
    i := 0
    while (i < c.tamaño)
        if (! pertence(c.datos[i], otro))
            sacar(c.datos[i], c)
        endif
        i := i + 1
    endwhile
endproc
```

c) i) **conjVacío** Todas las operaciones son $O(1)$ por lo que el algoritmo es $O(1)$

ii) **pertence** El algoritmo recorre todo el vector de datos por lo que es $O(n)$ donde n son los elementos del conjunto

iii) **agregar** Todas las operaciones son $O(1)$ por lo que el algoritmo es $O(1)$

iv) **sacar** Los peores casos son cuando el elemento a sacar es el primero del vector (hay que reubicar todos los demás elementos) o es el último (hay que recorrer todo hasta encontrarlo), por lo que el algoritmo es $O(n)$ donde n son los elementos del conjunto

v) **unir** En este caso tenemos:

- Un ciclo que recorre todos los elementos del conjunto `otro`, $O(m)$ donde m son los elementos de ese conjunto
- Dentro del ciclo llamamos a **pertence** y **agregar** sobre el conjunto `c`, que son $O(n)$ y $O(1)$ respectivamente.
- Las demás operaciones también son $O(1)$

El peor caso es cuando ningún elemento de `otro` está en `c` y entramos al if en todas las iteraciones. La complejidad entonces queda $O(m \times n + m \times 1) = O(m \times n)$

vi) **intersecar** En este caso tenemos:

- Un ciclo que recorre todos los elementos del conjunto `c`, $O(n)$
- Dentro del ciclo llamamos a **pertence** `otro`, que es $O(m)$ donde m son los elementos de ese conjunto
- En el peor caso, en cada ciclo llamaríamos a **sacar** en `c`, que es $O(n)$
- Las demás operaciones son $O(1)$

La complejidad entonces queda $O(n \times m + n \times n) = O(n^2 + m \times n)$

Observación: Ninguna de estas complejidades depende de la capacidad del conjunto

Ejercicio 2. Implementar el TAD Conjunto$\langle T \rangle$ (definido en el apunte de TADs) usando la siguiente estructura

```
Módulo ConjuntoLista<T> implementa Conjunto⟨T⟩ <
  var datos: ListaEnlazada<T>
  var tamano: int
>
```

a) Escriba el invariante de representación y la función de abstracción

b) Escriba los algoritmos para las operaciones `conjVacío` y `pertence`

i) **conjVacío**
ii) **pertence**
iii) **agregar**
iv) **agregarRapido**
v) **sacar**
vi) **unir**
vii) **intersecar**

c) Calcule la complejidad de cada una de estas operaciones

Diseño sobre árboles, *heaps* y *tries*

Ejercicio 3. Implementar los siguientes TADs sobre ABB. Calcule las complejidades de los procs en mejor y peor caso

a) Conjunto$\langle T \rangle$

b) Diccionario$\langle K, V \rangle$

c) ColaDePrioridad$\langle T \rangle$

Recalcule, si es necesario, las complejidades en peor caso de los algoritmos de los TADs considerando que se implementan sobre AVL en vez de ABB.

Ejercicio 4. Implemente el TAD ColaDePrioridad$\langle T \rangle$ utilizando heaps (implementados con arreglos).

Ejercicio 5. ¿Cómo haría para implementar una ColaDePrioridad ordenada por dos criterios? Por ejemplo, se quiere tener una cola de personas donde el criterio de ordenamiento es por edad y, en caso de empate, por apellido. Describa todos los cambios necesarios a la implementación del ejercicio anterior.

Ejercicio 6. Implemente el TAD Diccionario$\langle T \rangle$ utilizando *tries*.

Elección de estructuras

En esta sección nos concentraremos en la elección de estructuras y los algoritmos asociados. Cuando el enunciado diga "diseñe este módulo", salvo que indique lo contrario, sólo nos interesa:

1. La estructura
2. Su invariante de representación y su función de abstracción
3. Las operaciones con sus parámetros y su complejidad
4. Los algoritmos correspondientes

Ejercicio 7. Se desea diseñar un sistema para registrar las notas de los alumnos en una facultad. Al igual que en Exactas, los alumnos se identifican con un número de LU. A su vez, las materias tienen un nombre, y puede haber una cantidad no acotada de materias. En cada materia, las notas están entre 0 y 10, y se aprueban si la nota es mayor o igual a 7.

Materia, Alumno, Nota ES $\mathbb{Z}$
TAD Sistema {
obs Notas : dicc$\langle Materia, \text{dicc}\langle Alumno, Nota\rangle\rangle$
proc nuevoSistema() : $Sistema$
proc registrarMateria(inout $s : Sistema$, in $m : Materia$)
proc registrarNota(inout $s : Sistema$, in $m : Materia$, in $a : Alumno$, in $n : Nota$)
proc NotaDeAlumno(in $s : Sistema$, in $a : Alumno, m : Materia$) : $Nota$
proc cantAlumnosConNota(in $s : Sistema$, in $m : Materia, n : Nota$) : $\mathbb{Z}$
proc cantAlumnosAprobados(in $s : Sistema$, in $m : Materia$) : $\mathbb{Z}$
}

Dados m la cantidad de materias y n la cantidad de alumnos, se desea diseñar un módulo con los siguientes requerimientos de complejidad temporal:

- nuevoSistema O(1)
- registrarMateria en O(log m)
- registrarNota en O(log n + log m)
- notaDeAlumno en O(log n + log m)
- cantAlumnosConNota y cantAlumnosAprobados en O(log m)

Solución

Para poder acceder a las notas de un alumno en tiempo logarítimico, los guardamos como claves de un DiccionarioLog. Luego, como queremos poder acceder a sus notas (para definirlas y para obtenerlas) con orden logarítmico de la cantidad de materias, haremos para c/alumno otro diccionario logarítmico con las materias como claves. Con esto tenemos "resueltas" las complejidades de **registrarNota** y **notaDeAlumno**.

Para poder resolver las operaciones que piden las notas de una materia necesitamos acceder a la materia en tiempo logarítmico y, una vez que accedimos a la materia, obtener la cantidad de alumnos con una nota (o un rango de notas) en $O(1)$. Para eso podemos aprovechar que las notas van de 0 a 10 y guardar la cantidad de alumnos con c/nota en un vector de ese tamaño, lo que nos permite acceder a esa cantidad en $O(1)$

Módulo SistemaImpl **implementa** Sistema <

```
var notasPorAlumno: DiccionarioLog<Alumno, DiccionarioLog<Materia, Nota>>
var notasPorMateria: DiccionarioLog<Materia, Vector<int>>
```

>

pred InvRep($si : SistemaImpl$) {
$todosLosVectoresOncePosiciones(si.notasPorMateria.elems) \wedge_L$
$todasLasNotasCeroADiez(si.notasPorAlumno.elems) \wedge_L$
$materiasConsistentes(si.notasPorMateria.elems, si.notasPorAlumno.elems) \wedge_L$
$cantidadesDeNotasConsistentes(si.notasPorMateria.elems, si.notasPorAlumno.elems)$
}

pred todosLosVectoresOncePosiciones($npm : \text{dicc}\langle Materia, \text{seq}\langle\mathbb{Z}\rangle\rangle$) {
$(\forall m : Materia)\ (m \in npm \rightarrow_L |npm[m]| = 11)$
}

pred todasLasNotasCeroADiez($npa : \text{dicc}\langle Alumno, \text{dicc}\langle Materia, Nota\rangle\rangle$) {
$(\forall a : Alumno)\ (a \in npa \rightarrow_L (\forall m : Materia)\ (m \in npa[a] \rightarrow_L 0 \leq npa[a][m] \leq 10))$
}

pred materiasConsistentes($npm : \text{dicc}\langle Materia, \text{seq}\langle\mathbb{Z}\rangle\rangle, npa : \text{dicc}\langle Alumno, \text{dicc}\langle Materia, Nota\rangle\rangle$) {
$(\forall a : Alumno)\ (a \in npa \rightarrow_L (\forall m : Materia)\ (m \in npa[a] \rightarrow m \in npm))$
}

pred cantidadesDeNotasConsistentes($npm : \text{dicc}\langle Materia, \text{seq}\langle\mathbb{Z}\rangle\rangle, npa : \text{dicc}\langle Alumno, \text{dicc}\langle Materia, Nota\rangle\rangle$) {
$(\forall m : Materia)\ (m \in npm \rightarrow_L (\forall i : \mathbb{Z})\ (0 \leq i \leq 10 \rightarrow_L npm[m][i] = totalNotaIPorMateria(npa, m, i)))$
}

aux totalNotaIPorMateria($npa : \text{dicc}\langle Alumno, \text{dicc}\langle Materia, Nota\rangle\rangle, m : Materia, i : \mathbb{Z}$) : $\mathbb{Z}$ =
$\sum_{a \in \text{claves}(npa)} \text{IfThenElse}(m \in npa[a] \wedge_L npa[a][m] = i, 1, 0)$

Ejercicio 8. El TAD *Matriz infinita de booleanos* tiene las siguientes operaciones:

- *Crear*, que crea una matriz donde todos los valores son falsos.
- *Asignar*, que toma una matriz, dos naturales (fila y columna) y un booleano, y asigna a este último en esa coordenada. (Como la matriz es infinita, no hay restricciones sobre la magnitud de fila y columna.)
- *Ver*, que dadas una matriz, una fila y una columna devuelve el valor de esa coordenada. (Idem.)
- *Complementar*, que invierte todos los valores de la matriz.

Ejemplo de uso del módulo:

```
MatrizInfinita M := Crear()
bool b1 := Ver(M, 0, 0)
Asignar(M, 1, 3, False)
Asignar(M, 100, 5000, True)
bool b2 := M.Ver(100, 5000)
Complementar(M)
bool b3 := Ver(M, 394, 788)
bool b4 := Ver(M, 100, 5000)
```

Tras lo que deberíamos tener

```
b1 = False
b2 = True
b3 = True
b4 = False
```

Elija la estructura y escriba los algoritmos de modo que las operaciones *Crear*, *Ver* y *Complementar* tomen $O(1)$ tiempo en peor caso.

Ejercicio 9. Se desea diseñar un sistema de estadísticas para la cantidad de personas que ingresan a un banco. Al final del día, un empleado del banco ingresa en el sistema el total de ingresantes para ese día. Se desea saber, en cualquier intervalo de días, la cantidad total de personas que ingresaron al banco. La siguiente es una especificación del problema.

```
TAD IngresosAlBanco {
    obs totales : seq⟨ℤ⟩
    proc nuevoIngresos() : IngresosAlBanco {
        asegura { totalDia = ⟨⟩ }
    }
    proc registrarNuevoDia(inout i : IngresosAlBanco, in cant : ℤ) {
        requiere { i = I_0 ∧ cant ≥ 0 }
        asegura { i.totales == I_0.totales ++ ⟨cant⟩ }
    }
    proc cantDias(in i : IngresosAlBanco) : ℤ {
        asegura { res == |i.totales| }
    }
    proc cantPersonas(in i : IngresosAlBanco, in desde : ℤ, in hasta : ℤ) : ℤ {
        requiere {
            0 ≤ desde ≤ hasta ≤ |i.totales|
        }
        asegura {
            res = Σ_{j=desde}^{hasta} i.totales[j]
        }
    }
}
```

1. Dar una estructura de representación que permita que la función *cantPersonas* tome $O(1)$.
2. Calcular cómo crece el tamaño de la estructura en función de la cantidad de días que pasaron.
3. Si el cálculo del punto anterior fue una función que no es $O(n)$, piense otra estructura que permita resolver el problema utilizando $O(n)$ memoria.
4. Agregue al diseño del punto anterior una operación *mediana* que devuelva el último (mayor) día d tal que cantPersonas$(i, 1, d)$ $\leq$ cantPersonas$(i, d+1$,totDias$(i))$, restringiendo la operación a los casos donde dicho día existe.

Solución

Numeramos los días desde 0, igual que la secuencia: el día j corresponde a $i.totales[j]$. Llamamos n a la cantidad de días registrados.

1. Una primera idea un poco naive es precalcular todas las respuestas posibles. Para cada h entre 0 y n guardamos un vector con la suma de cada intervalo que termina justo antes del día h.

   Módulo IngresosAcumulados **implementa** IngresosAlBanco <

   ```
   /* sumas[h][d] = personas que ingresaron desde el día d */
   /* hasta el día h-1 (0 <= d <= h <= n) */
   var sumas: Vector<Vector<int>>
   ```

   > Veamos un poco las complejidades con esta estructura:

   - **cantPersonas**: devuelve `sumas[hasta][desde]`. Son dos accesos a vector, $O(1)$, tal como se nos pide.
   - **registrarNuevoDia**: si ya hay n días, hay que armar una fila nueva para las $n+2$ posiciones y nos cuesta $O(n)$.

2. La fila h tiene $h+1$ elementos. Con n días hay filas $0, \ldots, n$, así que la estructura guarda $1 + 2 + \cdots + (n+1) = \frac{(n+1)(n+2)}{2}$ enteros. El tamaño crece como $\Theta(n^2)$, es decir cuadrático en la cantidad de días.

3. Repensando un poco la estructura anterior, nos damos cuenta de que no necesitamos guardar todas las combinaciones ya que con las **sumas acumuladas** (sumas de prefijos) nos alcanza. La posición j tiene el total de personas que ingresaron en los primeros j días (los días $0, \ldots, j-1$). Con esto podemos calcular los ingresos de cualquier intervalo restando dos sumas acumuladas como:

   $$cantPersonas(i, desde, hasta) = acumulado[hasta] - acumulado[desde]$$

   Módulo IngresosImpl **implementa** IngresosAlBanco <

   ```
   /* acumulado[j] = personas que ingresaron en los primeros j días */
   /* acumulado[0] = 0 */
   var acumulado: Vector<int>
   ```

   >

   El vector tiene $n+1$ posiciones, así que la memoria es $O(n)$.

   ```
   proc nuevoIngresos(): IngresosImpl
     res := new IngresosImpl                  // O(1)
     res.acumulado := new Vector<int>         // O(1)
     res.acumulado.agregarAtrás(0)            // O(1)

     return res
   endproc

   proc registrarNuevoDia(i: IngresosImpl, cant: int)
     totalAnterior := i.acumulado.último()              // O(1)
     i.acumulado.agregarAtrás(totalAnterior + cant)     // O(1) amortizado
   endproc

   proc cantDias(i: IngresosImpl): int
     return i.acumulado.longitud() - 1        // O(1)
   endproc

   proc cantPersonas(i: IngresosImpl, desde: int, hasta: int): int
     return i.acumulado[hasta] - i.acumulado[desde]   // O(1)
   endproc
   ```

   Todas las operaciones son $O(1)$ ya que son accesos a posiciones del vector y operaciones aritméticas. `registrarNuevoDia` es $O(1)$ amortizado, por el costo de `agregarAtrás` en un `Vector`.

4. El enunciado de este ítem cuenta los días desde 1: `cantPersonas(i, 1, d)` $\leq$ `cantPersonas(i, d+1, totDias(i))` compara los primeros d días con el resto. Como en este ejercicio tomamos como convención contar los días desde 0, eso es $cantPersonas(i, 0, d) \leq cantPersonas(i, d, n)$: los días $0, \ldots, d-1$ contra los días $d, \ldots, n-1$.

   Para resolverlo, no hace falta cambiar la estructura. Sea $T = acumulado[n]$ el total de personas. Para d con $1 \leq d \leq n-1$:

   - $cantPersonas(i, 0, d) = acumulado[d]$
   - $cantPersonas(i, d, n) = T - acumulado[d]$

   Entonces la condición de la mediana es $acumulado[d] \leq T - acumulado[d]$, es decir, $2 \cdot acumulado[d] \leq T$.

   El ítem no pide ninguna complejidad, así que alcanza con probar todos los valores de d y quedarnos con el último que cumple la condición:

```
proc mediana(i: IngresosImpl): int
  n := i.acumulado.longitud() - 1          // O(1)
  total := i.acumulado[n]                  // O(1)
  res := 1                                 // O(1)
  d := 1                                   // O(1)
  while (d < n)                            // O(n) ciclos
    if (2 * i.acumulado[d] <= total)       // O(1)
      res := d                             // O(1)
    endif
    d := d + 1                             // O(1)
  endwhile

  return res
endproc
```

   El ciclo da $n-1$ vueltas de costo $O(1)$, así que `mediana` es $O(n)$.

   ¿Una vuelta de tuerca más?

   Podemos pensar que como **acumulado** es no decreciente, existe un valor k tal que todo $d \leq k$ cumple la condición $2 \cdot acumulado[d] \leq T$ y todo $d > k$ no la cumple. La mediana es ese k.

   Para encontrarlo no hace falta probar todos los d. Es como adivinar un número del 1 al 100 cuando nos responden "mayor" o "menor": en lugar de probar 1, 2, 3, ..., probamos el 50 y con la respuesta descartamos la mitad. Algo análogo a lo que hacemos en la ya conocida búsqueda binaria y sabemos que cuesta $O(\log n)$. Acá hacemos lo mismo al probar un d del medio. Si cumple la condición, k es ese d o uno mayor, y descartamos los de la izquierda. Si no la cumple, k es menor, y descartamos los de la derecha. Repetimos con la mitad que queda hasta que quede un solo valor.

   En el algoritmo, `izq` y `der` marcan el rango que todavía no descartamos. Mantenemos como "invariante" que `izq` siempre cumple la condición y `der` nunca.

```
proc mediana(i: IngresosImpl): int
  n := i.acumulado.longitud() - 1          // O(1)
  total := i.acumulado[n]                  // O(1)
  izq := 1                                 // O(1)
  der := n                                 // O(1)
  while (der - izq > 1)                    // O(log n) ciclos
    medio := (izq + der) / 2               // O(1)
    if (2 * i.acumulado[medio] <= total)   // O(1)
      izq := medio                         // O(1)
    else
      der := medio                         // O(1)
    endif
  endwhile

  return izq
endproc
```

   Por ejemplo, con $acumulado = \langle 0, 2, 5, 6, 10, 12, 18, 19, 20 \rangle$ ($n = 8$, $T = 20$):

   - `izq` $= 1$, `der` $= 8$. Probamos $d = 4$: $2 \cdot 10 = 20 \leq 20$, cumple. Ahora `izq` $= 4$.

- `izq` $= 4$, `der` $= 8$. Probamos $d = 6$: $2 \cdot 18 = 36 > 20$, no cumple. Ahora `der` $= 6$.
- `izq` $= 4$, `der` $= 6$. Probamos $d = 5$: $2 \cdot 12 = 24 > 20$, no cumple. Ahora `der` $= 5$.
- `izq` $= 4$ y `der` $= 5$ quedaron pegados entonces la mediana es 4. Los primeros 4 días suman 10 y el resto también suma 10.

Observación: también se puede lograr `mediana` en $O(1)$. Al agregar un día, T crece y $acumulado[d]$ no cambia para los días que ya estaban. Entonces, si un d cumplía $2 \cdot acumulado[d] \leq T$, lo sigue cumpliendo, y la mediana queda igual o aumenta. Si guardamos la mediana actual en la estructura, `registrarNuevoDia` la aumenta mientras el valor siguiente cumpla la condición. Como la mediana no puede superar n, en total aumenta a lo sumo n veces, así que `registrarNuevoDia` sigue siendo $O(1)$ amortizado.

Ejercicio 10. La Maderera San Blas vende, entre otras cosas, listones de madera. Los compra en aserraderos de la zona, los cepilla y acondiciona, y los vende por menor del largo que el cliente necesite.

Tienen un sistema un poco particular y ciertamente no muy eficiente: Cuando ingresa un pedido, buscan el listón más largo que tiene en el depósito, realizan el corte del tamaño que el cliente pidió, y devuelven el trozo que queda al depósito. Por otra parte, identifican a cada cliente con un código alfanumérico de 10 dígitos y cuentan con un fichero en el que registran todas las compras que hizo cada cliente (con la fecha de la compra y el tamaño del listón vendido).

Este sería el TAD simplificado del sistema:

```
Cliente ES string
TAD Maderera {
    proc comprarUnListon(inout m : Maderera, in tamaño : ℤ)
        //comprar en el aserradero un listón de un determinado tamaño

    proc venderUnListon(inout m : Maderera, in tamaño : ℤ, in cli : Cliente, in f : Fecha)
        // vender un listón de un determinado tamaño a un cliente particular en una fecha determinada

    proc ventasACliente(in m : Maderera, in cli : Cliente)
        // devolver el conjunto de todas las ventas que se le hicieron a un cliente
        // (para cada venta, se quiere saber la fecha y el tamaño del listón)
}
```

Se pide:

a) Escriba una estructura que permita realizar las operaciones indicadas con las siguientes complejidades:

   i) comprarUnListon en $O(\log(m))$
   ii) venderUnListon en $O(\log(m))$
   iii) ventasACliente en $O(1)$

   donde m es la cantidad de pedazos de listón que hay en el depósito

b) Escriba el algoritmo para la operación `venderUnListon`

Solución

RESUELTO EN CLASE 11 (Elección de estructuras II)

Ejercicio 11. Se quiere implementar el TAD Biblioteca que modela una biblioteca con su colección de libros. Se modela la biblioteca como una sola estantería de capacidad arbitraria, dentro de la cual cada libro ocupa una posición. La biblioteca cuenta con un registro de socios que pueden retirar y devolver libros en cualquier momento. Por restricciones del sistema que se utiliza, un socio no puede registrarse con un nombre de más de 50 caracteres.

Cuando la biblioteca adquiere un nuevo libro o cuando un libro es devuelto, éste es insertado en el primer espacio libre de la estantería. Es decir, si los lugares ocupados son 1, 2, 3, 4 y se presta el libro en la posición 2, al agregar un nuevo libro

al catálogo éste será ubicado en la posición 2. Cuando el libro que estaba originalmente en la posición 2 sea devuelto, será ubicado en la primera posición libre, que será la 5.

El TAD tiene las siguientes operaciones, para las que se nos piden las complejidades temporales de peor caso indicadas, donde

- L es la cantidad de libros en la colección
- r es la cantidad de libros que el socio en cuestión tiene retirados
- k la cantidad de posiciones libres en la estantería
- `proc agregarLibroAlCatálogo`(inout $b : Biblioteca$, in $l : idLibro$)
  - **Requiere**: {l no pertenece a la colección de libros de b}
  - **Descripción**: la biblioteca adquiere un nuevo libro, lo suma a su catálogo y lo pone en la estantería en el primer espacio disponible.
  - **Complejidad**: $O(log(k) + log(L))$
- `proc pedirLibro`(inout $b : Biblioteca$, in $l : idLibro$, in $s : Socio$)
  - **Requiere**: {s es socio de la biblioteca y el libro l no está entre los libros prestados}
  - **Descripción**: el socio pasa a retirar un libro que se retira de la estantería y se acumula en sus libros prestados.
  - **Complejidad**: $O(log(r) + log(k) + log(L))$
- `proc devolverLibro`(inout $b : Biblioteca$, in $l : idLibro$, in $s : Socio$)
  - **Requiere**: {s es socio de la biblioteca y el libro l está entre sus libros prestados}
  - **Descripción**: el socio pasa a devolver un libro que previamente había tomado prestado. Vuelve a la estantería en el primer espacio disponible.
  - **Complejidad**: $O(log(r) + log(k) + log(L))$
- `proc prestados`(in $b : Biblioteca$, in $s : Socio$) : Conjunto$\langle Libro \rangle$
  - **Requiere**: {s es socio de la biblioteca}
  - **Descripción**: este procedimiento retorna los libros que el socio tomó prestados de la biblioteca y aún no devolvió.
  - **Complejidad**: $O(1)$
- `proc ubicaciónDeLibro`(in $b : Biblioteca$, in $l : idLibro$) : $Posicion$
  - **Requiere**: {l pertenece a la colección de libros de b y no está prestado}
  - **Descripción**: obtiene la posición del libro en la estantería.
  - **Complejidad**: $O(log(L))$

Los datos se van a guardar en la siguiente estructura incompleta

```
  Socio es string; Libro, Posición es int

Módulo BibliotecaImpl implementa Biblioteca <

  /* Socios y los libros que tienen prestados */
  var socios: Diccionario<Socio, Conjunto<Libro>>

  /* Libros y su posición en la estantería */
  var catálogo: DiccionarioLog<Libro, Posición>

  /* Posiciones vacías en la estantería */
  var posicionesLibres: ColaDePrioridadLog<Posición>
>
```

a) Definir con qué estructura hay que implementar los diccionarios y conjuntos para cumplir las complejidades de las operaciones

b) Escribir los algoritmos mostrando cómo se cumplen las cotas de complejidad requeridas.

Solución

Módulo BibliotecaImpl **implementa** Biblioteca <

```
/* Socios y los libros que tienen prestados */
var socios: DiccionarioDigital<Socio, ConjuntoLog<Libro>>

/* Libros y su posición en la estantería */
var catálogo: DiccionarioLog<Libro, Posición>

/* Posiciones vacías en la estantería */
var posicionesLibres: ColaDePrioridadLog<Posición>

proc nuevaBiblioteca(): BibliotecaImpl
    res := new BibliotecaImpl
    res.socios := new DiccionarioDigital<Socio, ConjuntoLog<Libro>>
    res.catálogo := new DiccionarioLog<Libro, Posición>
    res.posicionesLibres := new ColaDePrioridadLog<Posición>

    return res
endproc

proc agregarLibroAlCatalogo(b: BibliotecaImpl, l: Libro)
    if (b.posicionesLibres.tamaño() > 0)                    // O(1)
        proximaPosicion := b.posicionesLibres.desencolar()  // O(log(k))
        b.catálogo.definir(l, proximaPosicion)              // O(log(L))
    else
        b.catalogo.definir(l, b.catalogo.tamaño() + 1)      // O(log(L))
    endif
endproc                                                     // O(log(k) + log(L))

proc pedirLibro(b: BibliotecaImpl, l: Libro, s: Socio)
    posicion := b.catálogo.obtener(l)                       // O(log(L))
    librosSocio := b.socios.obtener(s)                      // O(1)

    b.catálogo.borrar(l)                                    // O(log(L))
    librosSocio.agregar(l)                                  // O(log(r))
    b.posicionesLibres.encolar(posicion)                    // O(log(k))
endproc                                                     // O(log(L) + log(r) + log(k))

proc devolver(b: BibliotecaImpl, l: Libro, s: Socio)
    librosSocio := b.socios.obtener(s)                      // O(1)
    librosSocio.sacar(l)                                    // O(log(r))

    agregarLibroAlCatalogo(b, l)                            // O(log(k) + log(L))
endproc                                                     // O(log(r) + log(k) + log(L))

proc prestados(b: BibliotecaImpl, s: Socio)
    return b.socios.obtener(s)                              // O(1)
endproc

proc ubicacionDeLibro(b: BibliotecaImpl, l: libro)
    return b.catálogo.obtener(l)                            // O(log(L))
endproc
```

>

Ejercicio 12. Se quiere implementar el `TAD Agenda` que modela una agenda semanal donde se registran actividades. Cada actividad tiene un identificador, un horario de inicio y uno de finalización. No puede haber dos actividades con el mismo identificador. Para simplificar, las actividades sólo pueden comenzar y terminar en horarios en punto (por ejemplo, 21:00 hs) y terminan en un horario posterior a su inicio. Tampoco pueden empezar y terminar en días diferentes. Para contar la

cantidad de actividades que transcurren en un determinado horario no se debe tener en cuenta aquellas que finalizan en ese momento. En cada actividad se pueden agregar tags que permiten agrupar las distintas actividades por temáticas. Los tags tienen cómo máximo 20 caracteres.

Dadas las siguientes operaciones y de acuerdo a las complejidades temporales de peor caso indicadas, donde

- d es la cantidad de días registrados hasta el momento
- a la cantidad total de actividades
- `proc RegistrarActividad`(inout $ag : Agenda$, in $act : IdActividad$, in $día : Día$, in $inicio : Hora$, in $fin : Hora$)
  1. **Requiere**: la hora de fin sea mayor que la de inicio y la actividad no está actualmente registrada.
  2. **Descripción**: se agrega la actividad a la agenda, marcando en el día indicado la hora de inicio y finalización.
  3. **Complejidad**: $O(\log(a) + \log(d))$
- `proc VerActividad`(in $ag : Agenda$, in $act : IdActividad$) : $struct < día : Día, inicio : Hora, fin : Hora >$
  1. **Requiere**: la actividad debe estar registrada en la agenda.
  2. **Descripción**: se devuelven el día y horario en que se realiza la actividad.
  3. **Complejidad**: $O(\log(a))$.
- `proc AgregarTag`(inout $ag : Agenda$, in $act : IdActividad$, in $t : Tag$)
  1. **Requiere**: la actividad está registrada en la agenda y aún no tiene registrado ese tag.
  2. **Descripción**: se agrega el tag a la actividad indicada.
  3. **Complejidad**: $O(1)$.
- `proc HoraMásOcupada`(in $ag : Agenda$, in $d : Día$) : $Hora$)
  1. **Requiere**: el día indicado tiene al menos una actividad registrada.
  2. **Descripción**: se devuelve la hora que tiene más actividades registradas.
  3. **Complejidad**: $O(\log(d))$.
- `proc ActividadesPorTag`(in $ag : Agenda$, in $t : Tag$) : $Conjunto < IdActividad >$
  1. **Requiere**: true.
  2. **Descripción**: se devuelven las actividades que tienen registrado el tag t.
  3. **Complejidad**: $O(1)$.

1. Definir la estructura de representación del módulo `AgendaImpl`, que provea las operaciones solicitadas.
2. Escribir en castellano el invariante de representación.
3. Escribir los algoritmos de todas las operaciones, explicando cómo se cumplen las complejidades pedidas para cada una.

Solución

> **RESUELTO EN CLASE 11 (Elección de estructuras II)**

Ejercicio 13. Un instituto de cocina quiere organizar las inscripciones a su Escuela Culinaria de Invierno (ECI) donde ofrecen varias materias con capacidad limitada y nombres únicos de largo acotado. Los alumnos pueden intentar inscribirse a las materias y el sistema les informará si han quedado inscriptos o no (en caso que los inscriptos a la materia ya alcanzaran su capacidad). Si no pudieran inscribirse, igualmente quedan registrados en el sistema y serán agregados a la materia si algún inscripto se da de baja, por estricto orden de inscripción.

El sistema deberá implementar las siguientes operaciones en la complejidad pedida, donde a_m es la cantidad de alumnos inscriptos a la materia m. En todos los casos se puede asumir que la materia está registrada en el sistema. Materia es `string`. Alumno y Capacidad son `int`

- `proc abrirMateria`(inout $i : Inscripciones$, in $m : Materia$, in $c : Capacidad$)
  - **Descripción**: Abre la inscripción a una materia
  - **Complejidad**: $O(1)$

- `proc IntentarInscripcion`(inout $i : Inscripciones$, in $m : Materia$, in $a : Alumno$) : bool
  - **Descripción**: Intenta inscribir a un alumno a una materia a la que no está inscripto. Devuelve si el alumno quedó inscripto o no
  - **Complejidad**: $O(log(a_m))$
- `proc DarseDeBaja`(inout $i : Inscripciones$, in $m : Materia$, in $a : Alumno$)
  - **Descripción**: Saca al alumno de los inscriptos (donde ya debería estar) y agrega al siguiente de la lista de espera
  - **Complejidad**: $O(log(a_m))$
- `proc InscriptosMateria`(in $i : Inscripciones$, in $m : Materia$) : $Conjunto(Alumno)$
  - **Descripción**: Devuelve el conjunto de alumnos inscriptos en la materia
  - **Complejidad**: $O(1)$

a) Plantear la estructura de representación del módulo `ECIInscripcionesImpl`, que provea las operaciones mencionadas cumpliendo las complejidades requeridas.

b) Escribir el algoritmo de `DarseDeBaja`. Justifique **detalladamente** que se cumple la complejidad requerida.

Solución

a) Estructura:

```
Materia es string
Alumno, Capacidad es int

Módulo ECIInscripcionesImpl implementa Inscripciones <

  /* Para cada materia, cuántos alumnos puede tener inscriptos */
  var capacidad: DiccionarioDigital<Materia, Capacidad>

  /* Para cada materia, los alumnos inscriptos */
  var inscriptos: DiccionarioDigital<Materia, ConjuntoLog<Alumno>>

  /* Para cada materia, los alumnos en espera, en orden de inscripción */
  var enEspera: DiccionarioDigital<Materia, ColaSobreLista<Alumno>>

>
```

- Los tres diccionarios son `DiccionarioDigital` (implementados sobre *trie*) y tienen como claves los nombres de materias ya que tienen largo acotado, lo que hace que buscar y definir una materia cueste $O(1)$ en cualquiera de los tres.
- Los inscriptos de cada materia están en un `ConjuntoLog` con a_m elementos. Agregar y sacar cuestan $O(\log a_m)$ y `tamaño` cuesta $O(1)$.
- La lista de espera de cada materia es una cola sobre lista enlazada. Respeta el orden de inscripción y encolar, desencolar y `vacía` cuestan $O(1)$.

Suponemos que un alumno que está en la lista de espera de una materia no vuelve a intentar inscribirse en esa materia. Si no, quedaría dos veces en la cola.

En cuanto a las operaciones:

- **abrirMateria**: define la materia en los tres diccionarios y todo es en $O(1)$.
- **IntentarInscripcion**: obtiene la capacidad y los inscriptos de la materia, $O(1)$. Si el tamaño del conjunto de inscriptos es menor que la capacidad, agrega al alumno, $O(\log a_m)$, y devuelve true. Si no, obtiene la cola de espera de la materia, $O(1)$, encola al alumno, $O(1)$, y devuelve false. Nos queda una complejidad de $O(\log a_m)$ tal y como se pide.
- **InscriptosMateria**: obtiene los inscriptos de la materia, $O(1)$, y devuelve una referencia al conjunto (sin copiarlo), $O(1)$.

b) Algoritmo:

```
proc darseDeBaja(i: ECIInscripcionesImpl, m: Materia, a: Alumno)
  inscriptosMateria := i.inscriptos.obtener(m)   // O(1)
  esperando := i.enEspera.obtener(m)             // O(1)
  inscriptosMateria.sacar(a)                     // O(log a_m)

  // Si hay alumnos esperando entra el primero que se anotó
  if (!esperando.vacía())                        // O(1)
    siguiente := esperando.desencolar()          // O(1)
    inscriptosMateria.agregar(siguiente)         // O(log a_m)
  endif
endproc
```

Justificación de la complejidad:

- `inscriptos.obtener(m)` y `enEspera.obtener(m)`: en un **trie**, buscar cuesta $O(|m|)$. Como el largo de las materias está acotado, cada búsqueda es $O(1)$.
- `inscriptosMateria.sacar(a)`: el conjunto es un AVL con a_m elementos, así que sacar cuesta $O(\log a_m)$.
- `esperando.vacía()` y `esperando.desencolar()`: en una cola sobre lista enlazada son $O(1)$.
- `inscriptosMateria.agregar(siguiente)`: en ese momento el AVL tiene $a_m - 1$ elementos, así que agregar cuesta $O(\log a_m)$.
- Las asignaciones y la comparación del `if` son $O(1)$.

Total: $O(1) + O(1) + O(\log a_m) + O(1) + O(1) + O(\log a_m) = O(\log a_m)$.

Ejercicio 14. Como saben que somos muy buenos haciendo nuestro trabajo, nos piden ayuda desde un **Museo** para poder organizar las colecciones que poseen. En cada colección se registran las piezas que serán expuestas al público en algún momento. Una vez registradas, un curador analiza cada obra para poder determinar el año de su creación entre otras cosas. Los especialistas saben que las únicas obras realmente valiosas para sus colecciones datan aproximadamente desde el 4000 a.C. hasta el 2025 d.C. Finalmente, es de interés para el museo conocer cuáles son las obras que pertenecen a un año particular y cuál es la obra más antigua de cada una de sus colecciones para poder hacer difusión y atraer al público.

El sistema deberá implementar las siguientes operaciones en la complejidad pedida, donde c es la cantidad de colecciones registradas, p es la cantidad de piezas. *Colección*, *Pieza*, *Año* son *int*.

- `proc AbrirMuseo()` : $Museo$
  - **Descripción**: Inicia el museo sin colecciones ni piezas registradas
  - **Complejidad**: $O(1)$
- `proc PrepararColección`(inout $m : Museo$, in $c : Coleccion$, in $ps : Conjunto(Pieza)$)
  - **Descripción**: Se registra la colección con las piezas recibidas en ps sin las etiquetas de los años
  - **Requiere**: El conjunto contiene al menos una pieza
  - **Complejidad**: $O(log(c) + p.log(p))$
- `proc RegistrarPieza`(inout $m : Museo$, in $c : Coleccion$, , in $p : Pieza$, , in $a : Año$)
  - **Descripción**: Se registra el año de la pieza
  - **Requiere**: La pieza está registrada en la colección y el año está en el rango indicado
  - **Complejidad**: $O(log(c) + log(p))$
- `proc PiezasPorAño`(in $m : Museo$, in $a : Año$) : $Conjunto(Pieza)$
  - **Descripción**: Devuelve las piezas creadas en ese año
  - **Complejidad**: $O(1)$
- `proc KPiezasMásAntiguaPorColección`(in $m : Museo$, in $c : Coleccion$, in $k : int$) : $Conjunto\langle Pieza\rangle$
  - **Descripción**: Devuelve las k piezas más antiguas de la colección
  - **Requiere**: k toma valores entre 0 y la cantidad de piezas en la colección

- **Complejidad**: $O(log(c) + k.log(p))$

Se pide:

a) Plantear la estructura de representación del módulo `Museo` que permita cumplir con las complejidades requeridas. **Justificar** con palabras de forma concisa las operaciones **PrepararColección**, **PiezasPorAño** y **KPiezasMásAntiguaPorColección**.

b) Escribir el algoritmo de la operación `RegistrarPieza`. Justifique **detalladamente** que se cumple la complejidad obtenida.

Solución

Representamos los años antes de Cristo con números negativos. Por ej, el 4000 a.C. es -4000. Tomamos p como la cantidad total de piezas del museo. Suponemos que el año de cada pieza se registra una sola vez, y que **KPiezasMásAntiguaPorColección** se pide con k a lo sumo igual a la cantidad de piezas de la colección que ya tienen año.

a) Estructura:

```
Colección, Pieza, Año es int

Módulo MuseoImpl implementa Museo <
  /* Para cada colección, todas sus piezas */
  var piezas: DiccionarioLog<Colección, ConjuntoLog<Pieza>>

  /* Para cada colección, las piezas que ya tienen el año registrado, */
  /* ordenadas por año (la más antigua primero) */
  var ordenPorAño: DiccionarioLog<Colección, ColaDePrioridadLogMin<tupla<Año, Pieza>>>

  /* Vector de 6026 posiciones, una por año entre el 4000 a.C. y el 2025 d.C. */
  /* En la posición a + 4000 están las piezas del año a */
  var piezasPorAño: Vector<ConjuntoLog<Pieza>>

>
```

piezas y **ordenPorAño** son `DiccionarioLog` con las mismas claves. Las colas de **ordenPorAño** son `ColaDePrioridadLogMin` o sea la cola de prioridad del apunte, pero con un *min heap* en lugar de max (el apunte solo tiene la de max, `ColaDePrioridadLog`, suponemos que la de mínimo es igual y tiene las mismas complejidades). Sale primero la tupla de menor año, es decir, la pieza más antigua.

- **AbrirMuseo**: crea dos diccionarios vacíos y un vector de 6026 conjuntos vacíos. 6026 es una constante, así que es $O(1)$.
- **PrepararColección**: crea un `ConjuntoLog` vacío y agrega una por una las piezas de ps. Cada agregado cuesta $O(\log p)$, así que en total es $O(p \cdot \log p)$. Define la colección en **piezas** con ese conjunto, $O(\log c)$, y en **ordenPorAño** con un *heap* vacío, $O(\log c)$, porque las piezas todavía no tienen año. En total, $O(\log c + p \cdot \log p)$.
- **PiezasPorAño**: los años posibles están acotados, así que usamos un vector indexado por año. Accedemos a la posición $a + 4000$ en $O(1)$ y devolvemos una referencia al conjunto, $O(1)$.
- **KPiezasMásAntiguaPorColección**: busca la colección en **ordenPorAño**, $O(\log c)$. Desencola k veces el mínimo de su *heap*, $O(\log p)$ cada vez. Agrega cada pieza al conjunto resultado (un `ConjuntoLineal` ya que las piezas son distintas, así que `agregarRápido` cuesta $O(1)$) y guarda la tupla en una lista auxiliar para no perder los datos, $O(1)$. Al final vuelve a encolar las k tuplas para dejar la estructura como estaba, $O(\log p)$ cada una. En total, $O(\log c + k \cdot \log p)$.

b) Algoritmo:

```
proc registrarPieza(m: MuseoImpl, c: Colección, p: Pieza, a: Año)
  m.ordenPorAño.obtener(c).encolar(<a, p>, a)   // O(log c + log p)
  m.piezasPorAño[a + 4000].agregar(p)           // O(log p)
endproc
```

Justificación de la complejidad:

- `ordenPorAño.obtener(c)`: es un AVL con c claves, así que buscar cuesta $O(\log c)$.

- `encolar(<a, p>, a)`: el *heap* de la colección tiene a lo sumo tantos elementos como piezas tiene la colección, y eso es a lo sumo p. Encolar cuesta $O(\log p)$.
- `piezasPorAño[a + 4000]`: acceso a una posición de un vector, $O(1)$.
- `agregar(p)`: el conjunto del año es un AVL con a lo sumo p piezas. Agregar cuesta $O(\log p)$.

Total: $O(\log c) + O(\log p) + O(1) + O(\log p) = O(\log c + \log p)$.